

```
if (self = [super init]) {  
    //  
    //  
}  
return self;  
}
```

If a class is initialised properly, it returns a reference to self (hence the id type). However, if it fails, it returns nil. We've done that using this code in the initWithName: initialiser implementation:

```
initialiser:  
- (id)initWithName:(NSString *) n  
{  
    return [self initWithName:n andRate:0.0f];  
}
```

If you have multiple initialisers, and each of them has different parameters, you should chain them by ensuring that they all call a single initialiser that performs the call to the super class' initialiser. This initialiser is called the designated initialiser.

Protocols

A protocol declares a programmed interface or a set of methods that any adopting class can choose to implement. The defining class of the protocol is expected to call the methods in the protocol that are implemented by the adopting class. Let's examine the UIAlertView class to understand protocols in detail:

```
UIAlertView *alert = [[UIAlertView alloc]  
initWithTitle:@"Hello"  
message:@"This is an alert view"  
delegate:self  
cancelButtonTitle:@"OK"  
otherButtonTitles:nil];  
[alert show];
```

We are using the UIAlertView class by creating an instance of it and then calling its show method which displays an alert view with one button - OK. If you tap on the OK button, the alert view gets automatically dismissed. Additional buttons can also be displayed by setting the otherButtonTitles: parameter like:

```
UIAlertView *alert = [[UIAlertView alloc]
```